

# portfolio-web 배포 작업 보고서

개인 포트폴리오 앱 fork 배포·커스터마이징 작업 기록 + 기술 스택 분석 + 초보자용 재구축 개발 계획서

**작성일** 2026-09-09 (Upstream 자동 동기화·크롬 확장 도메인 수정 반영, 기술 스택 분석·개발 계획서 신규 추가)

**작업자** statclaude

**저장소** <https://github.com/statclaude/portfolio-web> (원본: hanjungwoo3/portfolio-web)

**배포 주소(웹)** <https://statclaude.github.io/portfolio-web/>

**추가 산출물** Android APK (개인용, 비공개 — app-debug.apk)

## 1. 개요

공개된 개인 포트폴리오 관리 웹앱(hanjungwoo3/portfolio-web)을 본인 GitHub 계정(statclaude)으로 fork 하여, 로컬 개발 중이던 커스텀 기능을 실제 인터넷에 배포하고, 배포 과정에서 발생한 여러 문제를 해결한 작업을 정리한다. 최종적으로 본인 전용 도메인(statclaude.github.io)에서 정상적으로 시세 데이터가 표시되고, Google Drive 연동을 통한 개인 데이터 저장/불러오기까지 동작하는 상태로 완료했다.

## 2. 저장소 및 배포 구조

- 로컬 작업 경로: C:\Users\stati\projects\portfolio-web (Windows PC)
- origin = <https://github.com/statclaude/portfolio-web.git> — 본인 fork, push 대상
- upstream = <https://github.com/hanjungwoo3/portfolio-web.git> — 원본, pull-only (절대 push 금지)
- 배포 명령: npm run deploy → 빌드 후 git push origin main && gh-pages -d dist -b gh-pages
- 배포 결과물은 GitHub Pages 를 통해 <https://statclaude.github.io/portfolio-web/> 에 정적 호스팅됨

## 3. 수행 과정

### 3.1 GitHub 푸시 인증 문제 해결

최초 배포 시도 시 Git Credential Manager 가 이전에 로그인했던 다른 계정(go2gogo, statclaud-star 등)으로 자동 연결되어 403 Permission denied 오류가 반복 발생했다.

|    |                                                                                       |
|----|---------------------------------------------------------------------------------------|
| 증상 | git push 시 "Permission to statclaude/portfolio-web.git denied to go2gogo" 등 403 오류 반복 |
| 원인 | 브라우저 기반 GCM 인증이 PC 에 남아있던 다른 Google/GitHub 로그인 세션을 자동으로 선택함                           |
| 해결 | GitHub Personal Access Token(repo 스코프) 발급 후 GCM 팝업의 "Token" 탭에 직접 붙여넣는 방식으로 전환하여 해결   |

추가로 gh-pages 실행 시 "Author identity unknown" 오류가 발생하여 git config --global user.email / user.name 을 설정해 해결했다.

### 3.2 배포된 사이트에서 시세 데이터가 표시되지 않던 문제 해결

최초 배포 후 삼성전자 등 일부를 제외한 대부분의 카드에 데이터와 배경 그래프가 표시되지 않는 문제가 있었다. 두 가지 원인이 겹쳐 있었다.

|      |                                                                                                                                                                                                                         |
|------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 원인 1 | .env.local 의 VITE_PROXY_URL=http://localhost:8787 값이 프로덕션 빌드에도 그대로 포함되어, 배포된 사이트가 존재하지 않는 로컬 주소로 API 를 요청하고 있었음                                                                                                         |
| 원인 2 | workers/proxy/src/index.ts, worker.js 의 ALLOWED_ORIGINS 목록에 원저작자 도메인(hanjungwoo3.github.io)만 등록되어 있어, 별도 프록시를 두더라도 statclaude.github.io 발 요청이 CORS 로 차단됨                                                                |
| 해결   | ① .env.production 파일을 새로 만들어 프로덕션 전용 VITE_PROXY_URL 지정<br>② ALLOWED_ORIGINS 를 https://statclaude.github.io 로 수정 ③ 본인 명의 Cloudflare Worker(portfolio-proxy.statclaude.workers.dev)를 새로 배포하여 연결 → 재배포 후 전 종목 데이터 정상 표시 확인 |

### 3.3 원저작자 저장소를 가리키던 하드코딩 링크 수정

설정 창 및 일부 컴포넌트에 원저작자 저장소(hanjungwoo3/portfolio-web)로 향하는 가이드/문서 링크가 하드코딩되어 있어, 본인 fork 에서도 원본 저장소로 연결되는 문제가 있었다.

- SettingsDialog.tsx — Cloudflare/Deno/로컬프록시 가이드, 크롬 확장 가이드, 커밋 이력 링크 등 statclaude/portfolio-web 으로 수정
- UsMarketTab.tsx, MobileSimpleView.tsx — 동일한 성격의 가이드/커밋 링크 수정
- 단, 크롬 확장 다운로드 링크(EXT\_RELEASE\_URL)는 예외로 원저작자 저장소를 그대로 유지 — GitHub Release 는 fork 시 자동으로 복사되지 않아, 본인 저장소로 바꾸면 404 가 발생하기 때문. 추후 본인 Release 를 만들면 함께 변경 가능

### 3.4 환율(EUR/JPY) 카드 클릭 시 야후로 연결되는 현상 조사

EUR/JPY 환율 카드를 클릭하면 토스가 아닌 Yahoo Finance 페이지로 연결되는 현상을 조사했다. 코드상 quoteUrl() 함수는 TOSS\_SYMBOL\_URL 에 매핑이 없는 종목에 대해 Yahoo Finance 로 자동 폴백하도록 설계되어 있었다.

토스의 환율 허브 페이지(tossinvest.com/indices/exchange-rate)로 매핑을 시도해 보았으나, 해당 페이지에 유로화·엔화 데이터 자체가 없어 실효성이 없다고 판단, 코드를 원상 복구하고 기존 Yahoo Finance 폴백 동작을 그대로 유지하기로 결정했다.

### 3.5 사이트 접근 제한 방안 검토

배포된 사이트를 본인 또는 허용된 사람만 접근하도록 제한하는 방법을 검토했다.

- GitHub Pages 자체 접근 제한: Free/Pro/Team 플랜에서는 불가능하며 GitHub Enterprise Cloud 에서만 지원됨을 확인
- 대안으로 Cloudflare Access(무료 최대 50 명)를 검토했으나, Cloudflare Pages 로 호스팅을 이전하거나 커스텀 도메인을 Cloudflare 앞단에 두는 추가 작업이 필요하여 결정을 보류함

### 3.6 Google Drive 연동 — 본인 전용 OAuth 클라이언트 설정

기존 코드에 하드코딩된 CLIENT\_ID 가 원저작자 소유의 Google OAuth 클라이언트였기 때문에, 배포된 도메인(statclaude.github.io)이 승인된 리디렉션 URI 목록에 없어 Google Drive 저장/불러오기 로그인 시 redirect\_uri\_mismatch 오류가 발생했다.

|      |                                                                                                                                                                                                                                                                     |
|------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1 단계 | 본인 Google Cloud 프로젝트를 새로 만들고, External + Testing 모드로 OAuth 동의 화면 구성, drive.appdata 스코프만 추가, 승인된 JavaScript 원본/리디렉션 URI 에 <a href="https://statclaude.github.io/">https://statclaude.github.io/</a> , <a href="http://localhost:5173/">http://localhost:5173/</a> 등록 |
| 2 단계 | 새로 발급받은 Client ID 로 src/lib/googleAuth.ts 의 CLIENT_ID 상수를 교체. 1차 시도에서는 편집·커밋 도구가 모두 "성공"으로 보고했음에도 실제 커밋된 코드에는 반영되지 않은 사실을 git show HEAD 로 직접 확인, 파일을 재확인 후 다시 편집·커밋하여 실제 반영을 검증함                                                                                    |
| 3 단계 | 재배포 후에도 "액세스 차단됨 — 테스트 중이며 승인된 테스터만 접근 가능" 오류 발생. 이는 OAuth 클라이언트가 Testing 상태라 사전 등록된 테스트 사용자만 로그인 가능하기 때문임을 확인하고, 우선 본인 계정을 테스트 사용자로 등록하여 정상 동작을 확인                                                                                                                 |
| 4 단계 | 여러 사용자가 등록 없이도 자유롭게 로그인해 쓸 수 있도록 하기 위해, OAuth 앱을 "프로덕션"으로 게시하기로 결정. 이를 위해 필요한 앱 도메인 정보(홈페이지 URL, 개인정보처리방침 URL, 서비스 이용약관 URL) 중 개인정보처리방침·이용약관 페이지가 없어 브랜딩 설정이 "미완성" 상태로 표시됨                                                                                          |
| 5 단계 | public/privacy.html, public/terms.html 신규 작성(수집 정보, Google Drive appdata 스코프 사용 범위, 데이터 삭제 방법, 문의처 등 명시) 후 배포. 해당 URL 들을 Google Cloud Console 브랜딩 페이지에 등록                                                                                                           |
| 6 단계 | "대상" 탭에서 앱 게시(프로덕션 전환) 완료. 이후 사전 등록 없이도 임의의 Google 계정으로 로그인하여 저장/불러오기가 가능한 상태로 확인됨 (원저작자 사이트와 동일한 사용성 확보)                                                                                                                                                           |

참고: 프로덕션 상태에서는 앱이 정식 심사(verification)를 받지 않았기 때문에 로그인 시 "Google 에서 확인하지 않은 앱입니다" 경고가 표시되며, 사용자가 "고급 → 이동(안전하지 않음)"을 눌러야 진행된다. 개인/소규모(100명 미만) 사용에는 문제가 없으며, 경고 자체를 없애려면 별도의 정식 심사 절차가 필요하다.

### 3.7 프록시 단일 장애점 해결 — Deno 프록시 추가

Google Drive 연동 작업 완료 후 한동안 정상 동작하다가, PC·모바일 양쪽에서 삼성전자·SK 하이닉스 등 일부 종목만 데이터가 표시되고 나머지는 비어있는 현상이 재발했다. 화면 상단에는 "공용 프록시 모두 사용량 소진 (1/1) — 갱신 중지" 메시지가 표시되었다.

|    |                                                                                                                                                                                                                                                                                                                             |
|----|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 원인 | 프록시 공급자를 본인 명의 Cloudflare Worker 1 개만 등록해둔 상태였는데, 토스가 해당 워커의 클라우드 egress IP 풀을 차단함. 원저작자도 동일한 문제를 겪었으며, 원본 앱은 애초에 Cloudflare·Vercel·Deno·Render·Netlify·Supabase 등 6개 공급자로 분산해 이 문제를 회피하도록 설계되어 있었음 (본인 배포에는 1개만 반영되어 있어 대체 경로가 없었음)                                                                                        |
| 해결 | 원저작자가 공개한 안내(GitHub 이슈/README)를 참고해 workers/deno-proxy/main.ts 의 ALLOWED_ORIGINS 를 statclaude.github.io 로 수정 → Deno Deploy 에 두 번째 프록시로 배포( <a href="https://portfolio-deno-proxy.statclaude.deno.net">https://portfolio-deno-proxy.statclaude.deno.net</a> ) → .env.production 에 VITE_PROXY_URL_2 로 등록 → 재배포 후 데이터 정상 표시 확인 |

코드 구조상 프록시 다운 판정은 최근 30 초간 연속 5 회 실패 기준의 롤링 윈도우이며, 한 번이라도 성공하면 즉시 정상 상태로 복귀하도록 되어 있어 별도의 수동 리셋 없이 자동 복구된다. 다만 등록된 공급자가 동시에 모두 막히면 자동 갱신 주기가 계속 느려지는 구조라, 향후 같은 현상이 잦아지면 세 번째 공급자 추가를 검토하기로 했다.

### 3.8 제휴 광고 배너 제거

메인 화면 상단에 원저작자의 카카오페이증권 제휴(추천인) 링크 배너("삼성전자 SK 하이닉스 주식 무료로 받기")가 항상 노출되고 있어 제거를 진행했다.

- App.tsx(PC 화면), MobileSimpleView.tsx(모바일 화면)에서 PromoBanner 컴포넌트 렌더링 및 관련 import 제거
- DonateDialog.tsx(개발자 후원 다이얼로그) 안에 있는 동일 배너는 "추후 재활용할 수 있다"는 판단하에 이번에는 그대로 유지하기로 결정

### 3.9 유로/엔화 카드가 토스(달러 정보만 있음)로 연결되던 문제 재조사 및 완전 해결

이전에 "유로/엔화를 토스 환율 허브 페이지로 연결해봤다가, 해당 페이지에 유로·엔화 데이터가 없어 원래대로 (야후 폴백) 되돌리기로" 결정했던 사안이 있었는데, 실제 배포된 사이트에서는 여전히 유로·엔화 클릭 시 토스로 연결되어 달러 정보만 보이는 현상이 재발했다.

|              |                                                                                                                                                                                                                                                           |
|--------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>확인</b>    | src/lib/toss.ts 의 TOSS_SYMBOL_URL 을 직접 점검한 결과, "EURKRW=X"·"JPYKRW=X" 항목이 여전히 토스 환율 허브 페이지로 매핑되어 있었음. 원저작자 저장소의 동일 파일과 직접 대조해, 원본에는 이 두 항목이 아예 없다는 것(KRW=X 만 존재)을 확인함                                                                                      |
| <b>근본 원인</b> | 지난번 "되돌리기" 작업 시, 클라우드 작업공간에서 수정한 파일이 원본과 바이트 단위로 동일함을 확인했으나, 이는 PC 로 커밋하기 전 로컬 편집 결과물을 확인한 것이었고, 커밋 도구가 "성공"으로 응답한 뒤 실제로 PC 에 반영된 파일을 다시 가져와 재확인하는 절차가 없었음. 이번 세션의 Google OAuth CLIENT_ID 건에서도 동일하게 "도구는 성공을 보고했지만 실제로는 반영 안 됨" 사례가 있었던 것과 같은 유형의 문제로 파악됨 |
| <b>해결</b>    | EURKRW=X, JPYKRW=X 매핑 두 줄을 제거하고 관련 주석을 정리 → 커밋 후 device 에서 파일을 다시 가져와(재-staging) 실제 내용을 직접 읽어 반영을 검증 → 배포 후 유로/엔화 클릭 시 야후로 정상 연결됨을 사용자가 확인함                                                                                                               |
| <b>재발 방지</b> | 이후로는 device 에 파일을 커밋할 때마다 도구의 성공 응답만으로 끝내지 않고, 매번 다시 파일을 가져와 실제 내용을 읽어 확인하는 절차를 표준 작업 방식으로 채택함                                                                                                                                                            |

## 4. Android 앱(APK) 버전 제작

### 4.1 배경 및 요구사항

원저작자가 자신의 배포 사이트(<http://5959.pages.dev/> → 실제로는 기존 <https://hanjungwoo3.github.io/portfolio-web/> 로 302 리다이렉트되는 링크였음)에 안드로이드 APK 를 추가로 배포했다는 소식을 전달받았다. 사용자는 다음 세 가지를 요청했다: (1) 현재 배포 중인 웹 버전은 그대로 유지 (2) 유로환율 기능을 포함한 새 앱(APK) 버전을 추가로 제작 (3) 실제 코드 작업 전에 원저작자의 앱 버전이 어떤 방식으로 개발됐는지 먼저 분석.

## 4.2 원저작자 앱 버전 아키텍처 분석

WebFetch 및 GitHub 저장소 조사를 통해, 이 "앱 버전"이 완전히 새로운 프로젝트가 아니라 기존 웹 PWA와 동일한 저장소/코드베이스를 Capacitor(네이티브 Android 래퍼 프레임워크)로 감싼 것임을 확인했다.

|                |                                                                                                                                                                                               |
|----------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 배포 형태          | GitHub Release(태그 app-v1.0.0, app-v1.0.1)로 APK 파일을 첨부해 배포. "Unknown sources" 허용 후 사이드로드 설치, iOS는 미지원                                                                                          |
| 핵심 원리          | 지금까지 프록시가 필요했던 이유는 IP 차단이 아니라 CORS 였다는 재발견. 토스·네이버가 Access-Control-Allow-Origin 헤더를 주지 않아 브라우저(웹)에서만 차단되는 것이고, 네이티브 HTTP(OkHttp)에는 CORS 자체가 없어 앱에서는 프록시 없이 직접 호출 가능                           |
| 핵심 파일          | capacitor.config.ts(신규), src/lib/nativeProxy.ts(신규, 네이티브 HTTP 전송 계층), src/lib/api.ts(네이티브 환경이면 최우선으로 직접 호출 후 실패 시 기존 프록시 체인으로 폴백하는 분기 추가), vite.config.ts(네이티브 빌드 시 base path·PWA 서비스워커 비활성화) |
| 최신 변경 (v1.0.1) | "원격 로드"로 전환 — APK 안에 웹 코드 사본을 내장하는 대신 capacitor.config.ts의 server.url로 배포된 gh-pages 사이트를 그대로 불러오도록 변경. 네이티브 프록시 기능은 URL과 무관하게 웹뷰에 주입되므로 그대로 동작하며, 이후로는 웹만 재배포하면 앱도 자동으로 최신 상태를 반영함            |

건너뛴 부분: 설정화면 APK 다운로드 안내 추가, gh-pages를 통한 APK 공개 배포/서명 관련 커밋들은 "공개 배포" 요소라 이번 작업 범위에서 제외했다 (4.3 참고).

## 4.3 개인용 한정 방침

원저작자와 달리, 완성된 APK를 웹사이트를 통해 공개 배포(GitHub Release, 설정 화면 다운로드 링크 등)하지 않고 본인만 사용하는 형태로 한정하기로 결정했다. 웹 버전은 기존과 동일하게 계속 최신화하되, APK 배포 관련 기능·링크·문서는 추가하지 않는다.

## 4.4 백업 및 작업 브랜치 준비

실제 코드 작업 전, 안전한 진행을 위해 물리적 백업과 별도 git 브랜치를 먼저 준비했다.

```
robocopy portfolio-web portfolio-web-backup-20260909 /E /XD node_modules dist /R:1 /W:1
git checkout -b android-app
```

robocopy로 프로젝트 폴더 전체(334개 파일, 약 15.45MB, node\_modules·dist 제외)를 별도 폴더에 복사했고, 기존 배포 중인 main 브랜치는 건드리지 않도록 android-app이라는 새 브랜치를 만들어 이후 모든 작업을 여기서 진행했다.

## 4.5 원저작자 커밋 선별 이식 (cherry-pick)

파일을 통째로 병합하는 대신, upstream 저장소를 fetch하여 실제로 어떤 커밋들이 이 기능을 추가했는지 커밋 히스토리를 먼저 조사한 뒤, 필요한 것만 선별적으로 가져왔다.

```
git fetch upstream git log upstream/main --oneline --grep="(앱)"
```

|        |                                                                                                                                   |
|--------|-----------------------------------------------------------------------------------------------------------------------------------|
| 가져온 커밋 | 53e705b(네이티브 프록시 기본 골격 + android/ 프로젝트 전체 스캐폴드), 2c28e82(야후 crumb 네이티브 처리), 135dbc1(구글 로그인 — 시스템 브라우저 + 딥링크 복귀)를 순서대로 cherry-pick |
| 부분 반영  | ff576ac(원격 로드 전환)는 커밋 전체가 아니라 capacitor.config.ts의 server.url 설정 부분만 수동으로 반영 — 같은 커밋에 포함된 공개 배포용 APK                              |

|        |                                                                                                           |
|--------|-----------------------------------------------------------------------------------------------------------|
|        | 바이너리(public/app/portfolio-app.apk)·release.json 은 제외                                                      |
| 제외한 커밋 | 2353cea(설정화면 APK 다운로드 안내), d0da8f1·6ea621b·78b9395(APK 서명·gh-pages 공개 배포 관련) — 전부 공개 배포 요소라 4.3 방침에 따라 제외 |

cherry-pick 과정에서 발생한 충돌과 해결:

|                                                      |                                                                                                                                                                                                                                                                                                                             |
|------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>public/<br/>version.json</b>                      | 빌드마다 자동 생성되는 커밋해시·타임스탬프 파일이라 내용이 무의미 — 들어오는 쪽 (theirs) 내용을 그대로 채택                                                                                                                                                                                                                                                           |
| <b>android/<br/>*.gradle,<br/>package.json<br/>등</b> | 본인 fork 에서 직접 손댄 적 없는 파일들 — 들어오는 쪽(theirs) 내용을 그대로 채택                                                                                                                                                                                                                                                                       |
| <b>src/lib/<br/>googleAuth.t<br/>s</b>               | 본인이 교체해둔 CLIENT_ID 상수와 원저작자의 신규 로직(시스템 브라우저 리다이렉트, 딥링크 처리)이 같은 위치에서 충돌. 원저작자의 새 import·상수 선언은 그대로 유지하면서 CLIENT_ID 값만 본인 것으로 유지하도록 수동 병합. 추가로 원저작자 코드에 하드코딩되어 있던 WEB_RELAY_URI(중계 페이지 주소)가 원저작자 본인 사이트(hanjungwoo3.github.io)로 되어 있던 것을 본인 사이트 (statclaude.github.io)로 수정 — 방치했다면 앱에서 구글 로그인 시 원저작자 사이트로 리다이렉트되는 오류로 이어질 뻔했음 |

## 4.6 원격 로드 설정 추가

ff576ac 커밋의 diff 만 확인하여, capacitor.config.ts 에 아래 설정을 URL 만 본인 도메인으로 교체해 수동으로 추가했다 (공개 배포용 APK 파일·release.json 은 제외).

```
server: { url: "https://statclaude.github.io/portfolio-web/", },
```

## 4.7 빌드 환경 구성 및 문제 해결

Android Studio 가 설치되어 있지 않은 상태에서 시작하여, 실제 APK 빌드까지 여러 버전 호환성 문제를 순서대로 해결했다.

|                            |                                                                                                                                                                |
|----------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>의존성 설치</b>              | npm install 시 vite-plugin-pwa@1.2.0 과 vite@8.0.10 간 peer dependency 충돌(ERESOLVE) 발생 — npm install --legacy-peer-deps 로 해결                                      |
| <b>Android Studio 설치</b>   | Android Studio Quail 3(2026 년 7 월 출시) 설치. Android SDK(Build-Tools, Platform)는 최초 빌드 시 Gradle 이 자동으로 추가 설치함                                                     |
| <b>JDK 버전 문제 1</b>         | Android Studio 에 내장된 JBR(JetBrains Runtime)이 Java 25 인데, 프로젝트의 Gradle Wrapper 버전(8.14.3)은 Java 24 까지만 지원 — "Unsupported class file major version 69" 오류로 빌드 실패 |
| <b>해결</b>                  | Eclipse Temurin JDK 17 을 설치 프로그램 없이 zip 으로 직접 받아(Adoptium API) 압축 해제 후 JAVA_HOME 을 그쪽으로 지정                                                                     |
| <b>local.properties 오류</b> | cmd 의 echo 명령으로 android/local.properties(SDK 경로 설정 파일)를 생성했을 때 보이지 않는 문자가 남아 "Trailing char" 오류 발생 — PowerShell 의 Set-Content 로 다시 생성해 해결                      |
| <b>JDK 버전 문제 2</b>         | JDK 17 로는 Capacitor 8.x 라이브러리가 요구하는 Java 21 소스 호환성을 만족 못해 "invalid source release: 21" 오류 발생                                                                   |
| <b>해결</b>                  | Eclipse Temurin JDK 21 로 교체(Gradle 8.14.3 이 지원하는 범위 — Java 21~24 — 안에서 Capacitor 가 요구하는 버전) 후 정상 빌드                                                            |



## 4.8 빌드 성공 및 설치

```
set JAVA_HOME=C:\tools\jdk-21.0.12.1+1 cd android gradlew assembleDebug
```

최종적으로 android/app/build/outputs/apk/debug/app-debug.apk (약 5.6MB) 빌드에 성공했다. 이 파일을 본인 폰으로 옮겨(USB 또는 이메일/클라우드 전송) "출처를 알 수 없는 앱" 설치를 허용해 사이드로드 설치했으며, 실행 시 statclaude.github.io/portfolio-web 라이브 사이트를 그대로 불러오면서 프록시 없이 직접 시세 데이터를 받아오고, 유로환율 카드도 정상적으로 표시되는 것을 확인했다.

## 4.9 appld 변경 — 원저작자 앱과의 설치 충돌 방지

cherry-pick 으로 가져온 capacitor.config.ts 와 android/app/build.gradle 의 applicationId 가 원저작자 원본 값(io.github.hanjungwoo3.portfolio) 그대로였다는 점을 확인했다. 안드로이드는 이 값(applicationId)으로 두 앱이 "같은 앱"인지 "다른 앱"인지 판단하므로, 방지할 경우 향후 원저작자가 정식 배포하는 앱을 같은 폰에 함께 설치하려 할 때 서명 불일치로 설치가 거부되거나 충돌할 가능성이 있었다.

|    |                                                                                                                                                                                                                    |
|----|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 검토 | android/app/build.gradle 에는 applicationId 외에 namespace(내부 R 클래스 패키지 경로)도 동일한 값을 쓰고 있었고, MainActivity.java 의 패키지 선언 ·strings.xml 의 custom_url_scheme(Google 로그인 딥링크 복귀와 연결된 값)도 같은 문자열을 참조하고 있었음                    |
| 결정 | 설치 충돌 여부를 실제로 결정하는 것은 applicationId 뿐이므로, namespace·MainActivity.java·strings.xml 은 그대로 두고 applicationId(및 대응하는 capacitor.config.ts 의 appld)만 본인 네임스페이스로 바꾸는 최소 위험 방식을 채택 — 정상 동작 중인 Google 로그인 딥링크 흐름을 건드리지 않기 위함 |
| 조치 | android/app/build.gradle 의 applicationId, capacitor.config.ts 의 appld 를 각각 "io.github.statclaude.portfolio"로 변경                                                                                                    |

변경 작업 중 부수적으로 발견된 문제: PowerShell 로 capacitor.config.ts(한글 주석 포함)를 인코딩 지정 없이 Get-Content/Set-Content 로 수정했더니 한글 주석이 깨지는 현상이 발생했다(Windows 기본 코드페이지로 읽고 쓰면서 UTF-8 텍스트가 손상됨). git checkout 으로 해당 파일을 원상 복구한 뒤, - Encoding UTF8 옵션과 [System.IO.File]::WriteAllText(UTF-8, BOM 없음)를 명시적으로 지정하는 방식으로 다시 수정해 한글 손상 없이 완료했다.

```
applicationId "io.github.statclaude.portfolio" // android/app/build.gradle
appId: "io.github.statclaude.portfolio", // capacitor.config.ts
```

이후 npx cap sync android 로 동기화하고 재빌드했다 — BUILD SUCCESSFUL in 30s. 새로 빌드된 APK 를 폰에 설치하니 appld 가 달라져 기존 앱과 별개의 새 앱으로 인식되었고(기존 앱을 지우지 않아도 동시 설치 가능한 상태), 사용자가 기존 구 appld 앱을 직접 삭제하고 새 앱이 정상 동작하는 것을 확인했다.

## 5. ETF Sector Flow(섹터별 흐름) 기능 반영

### 5.1 배경

원저작자가 지수 탭의 "🌱 한국 섹터 ETF" 고정 타일 그리드(개별 ETF 22 종 나열)와 별도의 반도체 TOP2+/소부장 고정 타일을 없애고, 전체 랭킹 스냅샷(825~1,127 종목)을 이름 기반으로 31 개 섹터로 묶어 보여주는 동적 컴포넌트로 교체했다는 것을 확인, 이 기능을 본인 fork 에 반영하기로 했다. 섹터 카드를 누르면 그 섹터의 ETF 목록 팝업이, 그 안에서 ETF 를 누르면 구성종목 팝업이 뜨는 3 단계 구조다.

### 5.2 커밋 선별 이식

원저작자 저장소에서 이 기능과 관련된 커밋 11 개(ce606ae ~ 0d774d9)를 조사해 feature/etf-sector-flow 브랜치에서 순서대로 cherry-pick 한 뒤 main 에 fast-forward 병합했다.

|            |                                                                                                                                                                                                                                          |
|------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 신규 파일      | src/lib/etfSectors.ts(31 개 섹터 이름 분류 규칙),<br>src/components/EtfSectorFlow.tsx(섹터 카드 그리드),<br>src/components/EtfSectorDialog.tsx(섹터→ETF 목록 팝업)                                                                                             |
| 기존 파일 변경   | dashboardGroups.ts("sector" 그룹에 render: "sectorFlow" 추가, 반도체 TOP2+/소부장 고정 타일 그룹 제거 — 데이터 자체는 etfSectors.ts 분류에 흡수되어 유지되고 화면의 고정 타일만 사라짐),<br>MobileSimpleView.tsx/UsMarketTab.tsx(동적 컴포넌트 렌더 분기),<br>etfRanking.ts(섹터 집계·캐시·자동조회 스로틀 로직) |
| 브랜치 사고와 복구 | 최초 작업 브랜치가 실수로 android-app 브랜치 위에서 생성된 것을 뒤늦게 발견 — git stash 와 브랜치 재생성으로 데이터 손실 없이 복구 후 11 개 커밋을 다시 적용                                                                                                                                   |
| 배포 정합성 문제  | 커밋되지 않은 파일 때문에 main 체크아웃이 실패한 상태에서 배포 스크립트를 실행해, gh-pages(실제 배포 사이트)는 새 내용으로 정상 갱신했지만 GitHub 의 main 브랜치 소스가 구버전으로 남는 불일치가 발생 — 원인 파악 후 정상적으로 main 을 병합·푸시하여 정합성 회복                                                                       |

조사 과정에서 원저작자가 더 최신 커밋(f497d2a 등)에서 이 기능을 ETF 기반에서 테마 바스켓 기반으로 한 단계 더 개편한 것을 발견했다. 사용자에게 채택 여부를 문의한 결과 "단추 이름만 바뀐 정도라면 추가로 반영하지 않아도 된다"는 결정을 받아, 현재 반영한 ETF 기반 버전에서 작업을 마무리했다.

### 5.3 배포 후 실제 동작 안 하던 문제 — 원인 규명 및 해결

코드 반영과 배포 자체는 문제없이 완료됐으나, 실제 라이브 사이트와 APK 양쪽에서 섹터 카드 자리가 계속 예전 고정 그리드(개별 ETF, 클릭 시 바로 토스로 이동)로만 표시되는 문제가 있었다. 세 단계로 원인을 좁혀 해결했다.

|           |                                                                                                                                                                                                                                                                                    |
|-----------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1 차 원인    | 개인 Deno 프록시(portfolio-deno-proxy.statclaude.deno.net)가 503 오류를 반환하고 있었음. Deno Deploy 대시보드 확인 결과, 조직이 결제수단으로 인증되지 않아 무료 플랜 한도의 1%(월 1 만 요청)만 허용되는 상태였고 그마저 소진되어 있었음(원래 무료 한도는 월 100 만 요청)                                                                                           |
| 1 차 해결    | Deno Deploy 대시보드에서 결제수단을 등록해 조직을 인증 → 한도가 정상 수준(월 100 만 요청)으로 복구되어 503 해소                                                                                                                                                                                                          |
| 2 차 원인    | 프록시를 복구한 뒤에도 라이브 사이트는 여전히 옛날 카드만 표시. 코드 확인 결과 섹터 카드의 자동조회는 브라우저에 개인 프록시 URL 이 등록돼 있는지 (localStorage)로만 판단하는데, 로컬 개발 서버(localhost)에서 등록해둔 값은 배포된 사이트(다른 origin)의 localStorage 에는 애초에 존재한 적이 없었음. 또한 섹터 데이터가 없는 폴백 상태에서는 수동으로 다시 조회할 새로고침 버튼 자체가 화면에 렌더링되지 않는 원저작자 코드의 UI 공백도 함께 확인됨 |
| 2 차 해결    | 배포된 사이트 자체의 설정 화면에서 개인 프록시 URL 을 새로 등록 → 정상적으로 섹터별 흐름(동적 카드)이 표시되는 것 확인                                                                                                                                                                                                            |
| 3 차 — APK | 동일한 증상이 APK 에서도 재현됨. APK 는 웹뷰로 같은 배포 사이트를 그대로 불러오는 구조라 실행 코드는 동일하지만, Android 웹뷰의 localStorage 는 PC/ 모바일 브라우저와도 별개의 저장소라 2 차에서 등록한 값이 APK 에는 없었음 → APK 안에서도 동일하게 설정 화면에서 개인 프록시 URL 을 등록해 PC 화면과 동일하게 정상 동작하는 것을 최종 확인                                                              |

이번 트러블슈팅을 통해, 이 앱의 개인 프록시 설정(localStorage 기반)은 "로컬 개발 서버 / 배포된 웹사이트(PC·모바일 브라우저) / APK" 세 실행 환경마다 저장소가 완전히 분리되어 있어 각각 따로 등록해야 한다는 점을 확인했고, 향후 유사 증상("한쪽 환경에서는 되는데 다른 환경에서는 안 된다")이 발생하면 이 가능성을 우선 점검하기로 했다.



## 6. Upstream 자동 동기화 알림 워크플로 구축

"자동 동기화"라는 이름이지만 자동 merge 는 하지 않는다 — 지금까지 원저작자 커밋을 선별적으로만 cherry-pick 해온 방식(appId·CLIENT\_ID·프록시 URL 등 본인 전용 값 유지, 광고 배너 제거·EUR/JPY 링크 수정처럼 의도적으로 다르게 만든 부분 유지, 테마 바스켓 리디자인처럼 알면서도 안 받기로 한 것도 있음) 과 자동 merge 는 근본적으로 상충하기 때문이다. 대신 "새 커밋이 생기면 알려주기"까지만 자동화하고, 실제 반영 여부·방법은 매번 직접 판단하는 것으로 설계했다.

- 파일: .github/workflows/upstream-sync.yml
- 동작: 매일 UTC 00:00(한국시간 09:00) + Actions 탭 수동 실행(workflow\_dispatch) 시 upstream 을 fetch 해, 지난 확인 시점(SHA 를 텍스트 파일로 저장) 이후 새 커밋이 있으면 그 목록을 GitHub 이슈로 생성. 새 커밋이 없으면 아무 것도 하지 않음. 첫 실행은 과거 이력이 한꺼번에 쏟아지는 것을 막기 위해 알림 없이 기준선만 저장하도록 설계

|      |                                                                                                                                                                                               |
|------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 문제 1 | git push 가 "refusing to allow a Personal Access Token to create or update workflow ... without workflow scope"로 거부됨 — 기존 PAT 가 repo 스코프만 갖고 있어서(.github/workflows/* 파일은 별도로 workflow 스코프가 필요) |
| 해결 1 | GitHub 토큰 설정 페이지에서 해당 Classic 토큰에 workflow 체크박스만 추가 — 재발급 없이 해결                                                                                                                               |
| 문제 2 | 저장소 Settings → Actions → General → Workflow permissions 가 기본값(읽기 전용)이라, 워크플로 자체는 실행되어도 이슈 생성·기준선 파일 커밋이 권한 부족으로 실패함                                                                           |
| 해결 2 | Workflow permissions 를 "Read and write permissions"로 변경                                                                                                                                       |
| 문제 3 | .github/workflows/ 경로는 원격 파일 전송 도구로 직접 쓸 수 없는 보호 경로라 자동으로 배포할 수 없었음                                                                                                                           |
| 해결 3 | 파일 내용을 사용자에게 그대로 전달해 VS Code 로 직접 생성 → git add/commit/push 는 사용자가 실행                                                                                                                          |

검증: Actions 탭에서 수동 실행 → 초록 체크(7 초) 성공 확인 → git pull 로 봇이 커밋한 기준선 파일이 로컬에 정상 반영되는 것까지 최종 확인해, 쓰기 권한 설정까지 전부 정상 작동함을 검증했다.

## 7. 크롬 확장 프로그램 도메인 수정

사용자의 질문("확장의 개선은 다운로드 링크만 바꾸는 것인가, 확장 자체를 수정해야 하는가")을 계기로 확장 소스(extension/)를 직접 분석한 결과, manifest.json 과 background.js 세 곳에 원저작자 도메인(hanjungwoo3.github.io)이 하드코딩되어 있어 statclaude.github.io 배포 사이트에서는 확장의 콘텐츠 스크립트가 아예 주입되지 않는다는 것을 확인했다. 사용자가 이 건에 한해 전권을 위임함에 따라, 코드 수정부터 패키징까지 대신 실행했다.

### 7.1 확장의 역할

- manifest.json — 권한 선언(declarativeNetRequest), 콘텐츠 스크립트가 주입될 페이지(content\_scripts.matches), 호스트 권한(host\_permissions)
- content.js — 앱 페이지와 서비스워커 사이를 window.postMessage 로 중계. 도메인과 무관하게 동작
- background.js — 시세 요청을 확장이 가정용 IP 로 직접 전송해 클라우드 프록시의 IP 차단·호출 한도를 회피, 토스 Origin 헤더를 declarativeNetRequest 로 재작성, 야후 crumb 인증 자체 처리, 톨바 아이콘으로 앱 탭 열기/포커스

## 7.2 수정 내역

|                           |                                                                                                                  |
|---------------------------|------------------------------------------------------------------------------------------------------------------|
| <b>manifest.json</b>      | version 1.0.0 → 1.1.0, content_scripts.matches 와 host_permissions 의 hanjungwoo3.github.io → statclaude.github.io |
| <b>background.js</b>      | APP_URL 상수를 statclaude.github.io/portfolio-web/ 로 변경                                                             |
| <b>extensionProxy.ts</b>  | EXPECTED_EXTENSION_VERSION 을 1.1.0 으로 동기화 (두 값이 어긋나면 패키징 스크립트가 버전 불일치로 막도록 이미 설계돼 있음을 확인)                        |
| <b>SettingsDialog.tsx</b> | EXT_RELEASE_URL 을 statclaude 저장소 릴리스 주소로 변경                                                                      |
| <b>부가 정리</b>              | OnboardingDialog.tsx·ProxyStatusBadge.tsx·ConsensusCharts.tsx·extension/README.md 에 남아있던 원저작자 저장소 가이드 링크도 함께 정리  |

의도적으로 손대지 않은 부분: src/lib/etfIndex.ts 의 ETF 데이터 소스 URL 은 링크가 아니라 실제 데이터 출처이므로 원저작자 저장소를 그대로 유지했다.

## 7.3 실행과 검증

이 세션은 사용자 PC 에서 셸 명령을 직접 실행할 수 없어, 코드 수정·npm run pack:extension 을 통한 v1.1.0 패키징(버전 일치 검사 통과 확인)까지는 클라우드 작업공간에서 수행하고, 수정된 파일 전체와 패키징된 zip 을 원격 파일 전송으로 PC 에 직접 반영한 뒤 다시 가져와 실제 내용을 검증했다. 다만 git commit/push(자격증명 필요)와 GitHub Release 생성 시 zip 첨부(브라우저 네이티브 파일 선택 다이얼로그가 필요해 원격 자동화로 채울 수 없음), 이 두 단계만 사용자가 직접 실행했다. 설치 후에는 Claude 내장 브라우저로 자동 검증을 시도했으나, 이 브라우저는 사용자의 실제 Chrome 과 분리된 별도 프로필이라 확장이 설치돼 있지 않아 검증에 사용할 수 없다는 것을 확인 — 대신 앱 자체의 상태 배지(설정 → 내 전용 프록시 → "🌿 크롬 확장 사용 중")로 확인하는 방법을 안내했고, 사용자가 실제 Chrome 에 설치한 뒤 정상 동작을 확인했다.

## 8. 최종 상태

- 웹 라이브 주소: <https://statclaude.github.io/portfolio-web/> — 정상 접속 및 전 종목 시세 데이터 표시 확인
- 환율 그룹에 유로환율(EURKRW=X) 카드 추가 완료
- 본인 명의 Cloudflare Worker + Deno 프록시 2 개 공급자 체제로 시세 데이터 이중화 확보
- 설정 화면 등 가이드 링크 대부분 본인 저장소 기준으로 수정 (확장 다운로드 링크, ProxyStatusBadge 가이드 링크만 예외)
- Google Drive 연동(백업/복원) — 본인 전용 OAuth 클라이언트로 교체, 프로덕션 게시 완료, 임의 사용자 가입 및 로그인 가능 상태로 확인
- 메인 화면(PC·모바일)의 제휴 광고 배너 제거 완료
- 유로/엔화 카드 클릭 시 야후 파이낸스로 정상 연결되는 것으로 최종 확인 완료
- Android APK(개인용, 비공개) 빌드 및 설치 완료 — 네이티브 프록시 우회 + 원격 로드 방식으로 웹과 동일한 최신 내용을 표시하며 정상 동작 확인
- APK 의 appId 를 원저작자 네임스페이스에서 본인 소유 네임스페이스 (io.github.statclaude.portfolio)로 변경 완료 — 추후 원저작자의 정식 배포 앱과 같은 폰에 동시 설치 가능한 상태로 확보
- ETF Sector Flow(섹터별 흐름) 기능 반영 완료 — 웹(PC·모바일 브라우저)과 APK 양쪽 모두에서 동적 섹터 카드가 정상 동작하는 것으로 최종 확인

- GitHub Actions 기반 upstream 자동 동기화 알림 워크플로 구축 완료 — 자동 merge 없이 새 커밋 발생 시 이슈로만 알림, 수동 실행까지 정상 동작 검증
- 크롬 확장 프로그램을 statclaude.github.io 전용(v1.1.0)으로 수정·패키징·릴리스하고, 실제 Chrome 설치 후 정상 동작 확인 완료

## 9. 남은 작업

- 사이트 접근을 본인/허용된 인원으로만 제한할지 여부 결정 (Cloudflare Access 등 — 보류 중)
- 프록시 공급자 동시 다운이 잦아질 경우 세 번째 공급자 추가 검토
- android-app 브랜치는 아직 main 에 병합하지 않은 상태 — 배포용 웹 브랜치(main)와는 분리 유지
- HeatmapTab.tsx·App.tsx 에 남아있는 원저작자 저장소 가이드 링크 정리 (이번 크롬 확장 작업 범위 밖으로 남겨둠)

## 10. 작업 원칙 변경

- 2026-09-08 부로, 프로그램에 실제 변경을 가하는 작업(코드 수정·커밋·배포 등)은 실행 전 항상 사용자에게 먼저 확인을 받은 후 진행하는 것으로 변경
- device 에 파일을 커밋한 뒤에는 도구의 성공 응답만 믿지 않고, 매번 다시 파일을 가져와 실제 반영 내용을 직접 읽어 검증하는 절차를 표준화
- 2026-09-09 부로, Android Studio 설치처럼 새 개발 환경이 필요한 작업은 사전 백업(폴더 복사 + git 브랜치 분리)을 먼저 마친 뒤 진행하는 것을 표준 절차로 채택
- APK 는 원저작자처럼 웹사이트를 통해 공개 배포하지 않는다 — 웹은 계속 최신화하되, APK 다운로드/배포 관련 기능·링크·문서는 추가하지 않고 본인만 빌드해 직접 사용
- 2026-09-09 부로, PowerShell 로 한글이 포함된 파일을 편집할 때는 Get-Content/Set-Content 에 -Encoding UTF8 을 반드시 명시하거나 [System.IO.File]::WriteAllText 로 UTF-8 을 직접 지정한다 — 인코딩 미지정 시 한글이 깨지는 문제 재발 방지
- 2026-09-09 부로, localStorage 기반 사용자 설정(개인 프록시 URL 등)은 로컬 개발 서버 / 배포된 웹사이트 / APK 세 실행 환경마다 저장소가 완전히 분리된다는 점을 확인 — "한쪽 환경에서는 되는데 다른 환경에서는 안 된다"는 증상이 나오면 코드·배포 문제이기 전에 이 가능성부터 점검하는 것을 표준 절차로 채택
- 2026-09-09 부로, .github/workflows/\*.yml 처럼 자격증명(PAT)이나 브라우저 네이티브 파일 다이얼로그가 필요한 단계는 사용자가 특정 작업 전체를 위임("전부 실행해줘")하더라도 대행할 수 없다는 것을 확인 — 이런 경우 나머지(코드 수정·패키징·검증)는 전부 대신 실행하고, 자격증명·파일첨부 단계만 정확한 절차로 안내해 사용자가 직접 실행하게 하는 것을 표준으로 채택
- 2026-09-09 부로, Claude 의 내장 브라우저는 사용자의 실제 Chrome 과 완전히 분리된 별도 프로필이라 크롬 확장이 설치돼 있지 않다는 것을 확인 — 확장 관련 동작 검증에는 사용할 수 없고, 사용자의 실제 Chrome 을 통한 확인(또는 실제 Chrome 이 연결되어 있을 때의 원격 자동화)만 유효하다는 것을 원칙으로 채택

## 11. 개발·배포·유지관리 항목별 기술/SW/지식 분석

본 프로젝트를 처음부터 지금 상태까지 진행하는 데 실제로 필요했던 항목을 영역별로 정리한다. 각 영역은 "무엇을 위해 필요했는가 → 구체적으로 쓰인 기술/SW → 그 기술을 다루기 위해 필요한 배경 지식" 순으로 기술한다.

## 11.1 프론트엔드 개발

- 목적: 종목 카드·차트·설정 화면 등 사용자가 보는 앱 화면 전체 구현
- 기술/SW: React 18, TypeScript, Vite(빌드 도구), Tailwind CSS, @tanstack/react-query(서버 상태 캐싱), Node.js/npm, VS Code
- 필요 지식: 컴포넌트·훅(hook) 기반 설계, 클라이언트 상태 vs 서버 상태 구분, 반응형(모바일/PC) 레이아웃, PWA(서비스워커·매니페스트) 기본 개념, 브라우저 개발자도구 활용

## 11.2 데이터 수집·프록시 인프라

- 목적: 토스·네이버·야후 등 외부 시세 API 를 CORS·IP 차단 없이 안정적으로 호출
- 기술/SW: Cloudflare Workers, Deno Deploy, Node.js 로컬 프록시 서버, Chrome 확장 (Manifest V3, declarativeNetRequest, 서비스워커)
- 필요 지식: CORS 의 정확한 동작 원리(브라우저만 강제, 서버·네이티브 클라이언트는 무관), HTTP 요청 헤더 조작(Origin/Referer), IP 차단(egress 풀 단위 차단) 개념, 서버리스 함수 배포 경험, 장애 감지·자동 폴백(다중 공급자) 설계

## 11.3 인증·클라우드 연동

- 목적: 보유 종목·거래 기록을 개인 Google Drive 에 백업/복원
- 기술/SW: Google OAuth 2.0, Google Drive API(appdata 스코프), Google Cloud Console
- 필요 지식: OAuth 인가 코드 흐름, 리디렉션 URI·승인된 origin 등록, 테스트/프로덕션 게시 절차와 심사(verification) 개념, 개인정보처리방침·이용약관 페이지 작성

## 11.4 네이티브 모바일(Android APK)

- 목적: 같은 웹 코드를 스마트폰에 설치형 앱으로 제공, 네이티브 HTTP 로 CORS 자체를 우회
- 기술/SW: Capacitor(웹뷰 네이티브 래퍼), Android Studio, Android SDK, Gradle, JDK(버전별 호환성 관리)
- 필요 지식: 웹뷰와 브라우저의 차이(저장소·네트워크 스택이 다름), 네이티브 HTTP 클라이언트 개념, appld/네임스페이스·서명이 앱 동일성을 결정하는 원리, Java/Gradle 빌드 시스템의 버전 호환 범위 읽는 법

## 11.5 브라우저 확장

- 목적: 설치한 사용자의 가정용 IP 로 직접 요청을 보내 클라우드 프록시의 차단·한도 문제를 근본적으로 우회
- 기술/SW: Chrome Extension Manifest V3, declarativeNetRequest API, 서비스워커, postMessage 기반 페이지-확장 통신
- 필요 지식: 확장 권한 모델(host\_permissions·content\_scripts 매칭 규칙), 헤더 재작성이 필요한 이유(교차출처 API 의 Origin 검증), 개발자 모드 확장의 수동 업데이트 특성

## 11.6 버전 관리·협업 워크플로

- 목적: 원저작자 저장소의 신규 기능을 필요한 것만 선별적으로 반영하면서 본인 커스터마이징을 안전하게 유지
- 기술/SW: Git, GitHub(fork/PAT), GitHub CLI

- 필요 지식: fork·upstream 개념, cherry-pick 으로 커밋 단위 선별 이식, 브랜치 분리 전략, 병합 충돌 해결, Personal Access Token 스코프 체계

### 11.7 배포·자동화(CI/CD)

- 목적: 코드 변경을 실제 서비스에 반영하고, 원저작자 저장소 변경 사항을 놓치지 않고 파악
- 기술/SW: GitHub Pages, gh-pages npm 패키지, GitHub Actions(workflow YAML)
- 필요 지식: 정적 사이트 배포 파이프라인, GitHub Actions 의 스케줄/수동 트리거·권한 (GITHUB\_TOKEN) 모델, 셸 스크립트 기초

### 11.8 유지관리·문서화

- 목적: 반복되는 문제(원인이 같은 재발 증상)를 빠르게 진단하고, 작업 이력을 남겨 재현 가능하게 관리
- 기술/SW: 상태 배지·사용량 카운터 같은 자체 모니터링 UI, docx(Node 라이브러리)·LibreOffice(PDF 변환)를 이용한 보고서 자동 생성
- 필요 지식: 트러블슈팅 방법론(증상-원인-해결 구조화), "도구가 성공을 보고해도 실제 반영을 재확인한다" 같은 검증 습관, 변경 이력을 남기는 문서화 습관

## 12. 초보자 관점 역순 개발 계획서 — 지금 상태를 처음부터 만든다면

프로그래밍을 처음 배우는 사람이 오늘의 완성된 상태(웹+APK+확장+자동화)를 목표로 삼는다면, 처음부터 이 모든 것을 한 번에 만들려고 해서는 안 된다. 아래는 실제로 이 프로젝트가 겪은 문제들의 순서를 거꾸로 짚어, "가장 단순한 것부터 만들고, 그 단순한 버전이 부딪히는 한계를 하나씩 해결해나가는" 방식으로 재구성한 단계별 계획이다. 각 단계는 이전 단계가 실제로 동작하는 상태를 전제로 한다.

### 12.0 단계 — 사전 학습 (본격 개발 전 필요한 최소 지식)

- HTML/CSS/JavaScript 기초, 그리고 JS 비동기 처리(Promise/fetch)에 대한 이해
- React 기초(컴포넌트, props, state, useEffect) — 공식 튜토리얼 수준으로 충분
- Git·GitHub 기초(commit/push/pull, 저장소 개념) — 이후 모든 단계에서 필요

이 단계의 목표는 완벽한 숙달이 아니라 "다음 단계를 따라갈 수 있는 최소한"이다. 나머지 지식(OAuth, CORS, Capacitor 등)은 각 단계에서 실제로 필요해질 때 그때그때 익히는 것이 효율적이다 — 이번 프로젝트도 실제로 그런 순서로 지식을 쌓았다.

### 12.1 단계 — 가장 단순한 MVP: 수동 입력형 포트폴리오 앱

외부 API 연동 없이, 종목명·수량·매입가를 사용자가 직접 입력하고 브라우저의 localStorage 에만 저장하는 정적 웹앱부터 만든다. 실시간 시세, 로그인, 배포 전략 등은 전부 뒤로 미룬다.

- 구현 범위: 입력 폼, 목록 표시, 간단한 손익 계산(수동 입력 매입가 기준), localStorage 저장/불러오기
- 왜 여기서 시작하는가: 외부 API·CORS·프록시 같은 이 프로젝트의 가장 어려운 부분들을 전부 배제한 채로, "React 컴포넌트 + 로컬 상태 저장"이라는 핵심 골격만 먼저 완성해 성취감과 구조적 기반을 동시에 확보하기 위함

### 12.2 단계 — 정적 배포로 "내 것"을 인터넷에 올리기

- 구현 범위: GitHub 저장소 생성 → GitHub Pages 로 12.1 단계 결과물 배포

- 왜 이 시점인가: 로컬에서만 도는 앱과 "인터넷에 실제로 존재하는 앱"은 완전히 다른 경험이다. 데이터 연동을 붙이기 전에 배포 파이프라인 자체를 먼저 익혀두면, 이후 단계마다 매번 새로 배우지 않고 재배포만 반복하면 된다

### 12.3 단계 — 실시간 시세 연동 시도 → CORS 벽 학습

- 구현 범위: 종목 하나에 대해 외부 시세 API 를 fetch 로 직접 호출해보기
- 여기서 반드시 마주치는 문제: 브라우저 콘솔에 CORS 오류가 뜨며 요청이 차단됨
- 왜 일부러 이 실패를 거치는가: 이 프로젝트 전체의 핵심 난제(CORS·IP 차단)를 초반에 직접 겪어보아, 이후 프록시·네이티브 앱·브라우저 확장이라는 세 가지 우회 전략이 "왜 각각 존재하는지"를 체감으로 이해할 수 있다

### 12.4 단계 — 가장 단순한 우회: 내 PC 에서 도는 로컬 프록시

- 구현 범위: Node.js 로 몇 줄짜리 로컬 서버를 만들어, 앱이 이 서버에 요청 → 서버가 대신 외부 API 를 호출해 응답을 돌려주는 구조(프록시)를 구현
- 왜 클라우드가 아니라 로컬부터인가: 서버리스 배포(Cloudflare 등)의 계정 생성·설정 절차를 아직 모르는 상태에서, "프록시가 왜 필요하고 어떻게 동작하는지"라는 개념 자체를 가장 적은 도구로 먼저 이해하기 위함

### 12.5 단계 — 상시 구동되는 클라우드 프록시로 전환

- 구현 범위: Cloudflare Workers(또는 Deno Deploy) 무료 플랜에 4 단계와 동일한 역할의 프록시를 배포, 배포된 웹앱이 이 주소를 사용하도록 전환
- 왜 이 시점인가: 로컬 프록시는 내 PC 를 켜둔 동안만 동작하므로, 배포된 사이트를 다른 사람(또는 다른 기기)이 접속했을 때는 무용지물이다. "배포된 프론트엔드 + 상시 구동 백엔드"라는 조합을 처음으로 완성하는 단계

### 12.6 단계 — 단일 장애점 제거: 프록시 이중화

- 구현 범위: 서로 다른 공급자(예: Cloudflare + Deno)에 같은 역할의 프록시를 하나씩 더 배포하고, 앱이 순서대로 시도하다 실패하면 다음 공급자로 자동 전환하는 로직 추가
- 왜 이 시점인가: 5 단계까지 오면 "프록시 하나가 막히면 서비스 전체가 멈춘다"는 한계가 실제로 체감된다 — 이중화는 이 문제를 실제로 겪은 뒤에 배워야 필요성이 와닿는 전형적인 사례

### 12.7 단계 — 사용자 데이터 클라우드 백업

- 구현 범위: Google Cloud Console 에서 OAuth 클라이언트 생성 → Google Drive API 로 로그인·저장·불러오기 구현 → 처음엔 테스트 모드로, 이후 프로덕션 게시
- 왜 이 시점인가: localStorage 만으로는 기기를 바꾸거나 브라우저 데이터가 지워지면 기록이 사라진다. 이 문제 역시 "실제로 데이터가 소중해지는 시점"(어느 정도 실사용을 하며 데이터가 쌓인 뒤)에 배우는 것이 동기부여 면에서 자연스럽다

### 12.8 단계 — 도메인 기능 확장

- 구현 범위: 환율/지수 카드, ETF 랭킹·섹터별 흐름, 재무제표 컨센서스 차트 등 앱의 실제 "기능"을 하나씩 추가
- 왜 이 시점인가: 1~7 단계로 "데이터가 안정적으로 들어오고 어딘가에 안전하게 저장되는" 기반 인프라가 갖춰진 뒤에야, 그 위에 기능을 쌓는 작업이 반복적인 인프라 디버깅 없이 순조롭게 진행된다



## 12.9 단계 — 신뢰성 강화: 브라우저 확장으로 우회 채널 추가

- 구현 범위: 최소 권한(declarativeNetRequest, 필요한 호스트만)의 Chrome 확장을 별도로 만들어, 설치한 사용자의 가정용 IP 로 직접 요청을 보내는 채널 추가
- 왜 이 시점인가: 클라우드 프록시를 아무리 이중화해도 공급자 정책 변경·사용량 한도라는 근본적 한계는 남는다 — 이 한계를 실제로 여러 번 겪은 뒤에야 "프록시가 아예 필요 없는 방법"을 찾게 되는 순서가 자연스럽다

## 12.10 단계 — 모바일 앱화: PWA 에서 네이티브 APK 로

- 구현 범위: 먼저 PWA(웹 매니페스트 + 서비스워커)로 "홈 화면에 추가" 수준까지 만들고, 이후 여유가 있다면 Capacitor 로 같은 코드를 네이티브 래핑해 APK 로 빌드
- 왜 이 순서인가: PWA 는 기존 웹 코드에 설정 파일만 추가하면 되어 진입장벽이 낮고, Capacitor·Android Studio·Gradle·JDK 버전 관리처럼 상대적으로 복잡한 네이티브 빌드 환경은 "웹 버전이 충분히 안정된 뒤" 별도 학습 곡선으로 다뤄야 좌절하지 않는다

## 12.11 단계 — 유지관리 체계화

- 구현 범위: 원본 저장소를 계속 추적하고 있다면 GitHub Actions 로 원본의 신규 커밋을 주기적으로 확인해 알려주는 자동화 추가, 변경 이력을 문서로 남기는 습관 정착
- 왜 마지막 단계인가: 지금까지 쌓인 커스터마이징(appld·CLIENT\_ID·제거한 배너 등)을 자동 병합이 덮어쓰지 않도록, "무엇을 의도적으로 다르게 유지하고 있는지"가 충분히 쌓인 뒤에야 자동화 규칙을 제대로 설계할 수 있다

## 12.12 단계별 핵심 조언

- 각 단계는 이전 단계가 실제로 동작하는 채로 다음 단계를 시작한다 — 여러 단계를 동시에 진행하면 문제가 생겼을 때 원인을 좁히기 어렵다(이번 프로젝트에서도 배포 정합성 문제·브랜치 사고 등은 대부분 여러 작업을 한 번에 진행하다 발생했다)
- 에러 메시지·콘솔 로그를 직접 읽고 원인을 좁히는 습관이 새 프레임워크 지식보다 더 중요하다 — 실제로 이 프로젝트의 문제 대부분(CORS, redirect\_uri\_mismatch, appld 충돌, localStorage 분리 등)은 정확한 에러 메시지를 읽고 원인을 좁혀가는 과정에서 해결됐다
- "도구가 성공했다고 응답했다"와 "실제로 반영됐다"는 다르다 — 특히 원격 환경에 파일을 배포하는 작업에서는 반드시 재조회로 검증하는 습관을 처음부터 들이는 것이 좋다